

Module 7. Unsupervised learning

COMP 551 Applied machine learning

Yue Li

Assistant Professor
School of Computer Science
McGill University

April 8 and 10, 2025

Logistics

Mercury Course Evaluation open until **April 30**.

Please fill out the course evaluation. It means a lot to me 😊

Outline

Learning objectives

Unsupervised learning

K-means clustering

Principal component analysis

Autoencoders (optional)

- MLP autoencoder

- Denoising autoencoder

- CNN autoencoder

Evaluating unsupervised learning methods

Summary

Final exam (April 16 9-noon)

Learning objectives

- K-means clustering
- Principal component analysis
- Autoencoder (optional)
- Evaluating unsupervised learning

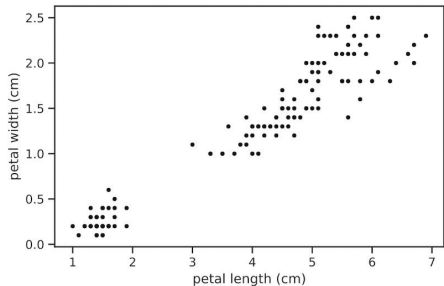
Unsupervised learning

- Instead of having a pair of input and output in the supervised learning tasks (Modules 1-6), in **unsupervised learning**, we seek to find latent patterns from *only the inputs* data $\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^N$ without any corresponding output y_n .
- Unsupervised learning forces the model to “explain” the high-dimensional inputs, rather than just the low-dimensional outputs (i.e., trying to “make sense of” the data)
- The reason we need unsupervised learning is that most of the data we have are *unlabelled*.

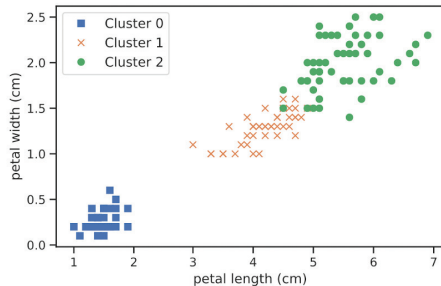
“Unsupervised learning forces the model to make sense of the data.” – Geoffrey Hinton

Clustering

2D data for the 149 iris flowers

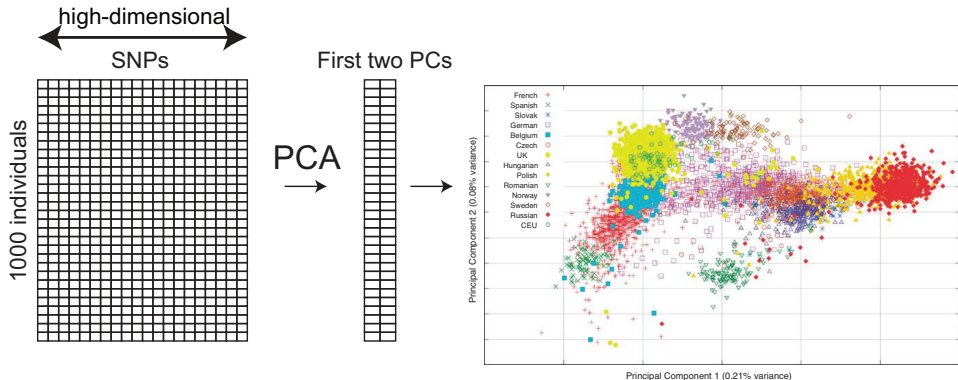


K-means
→

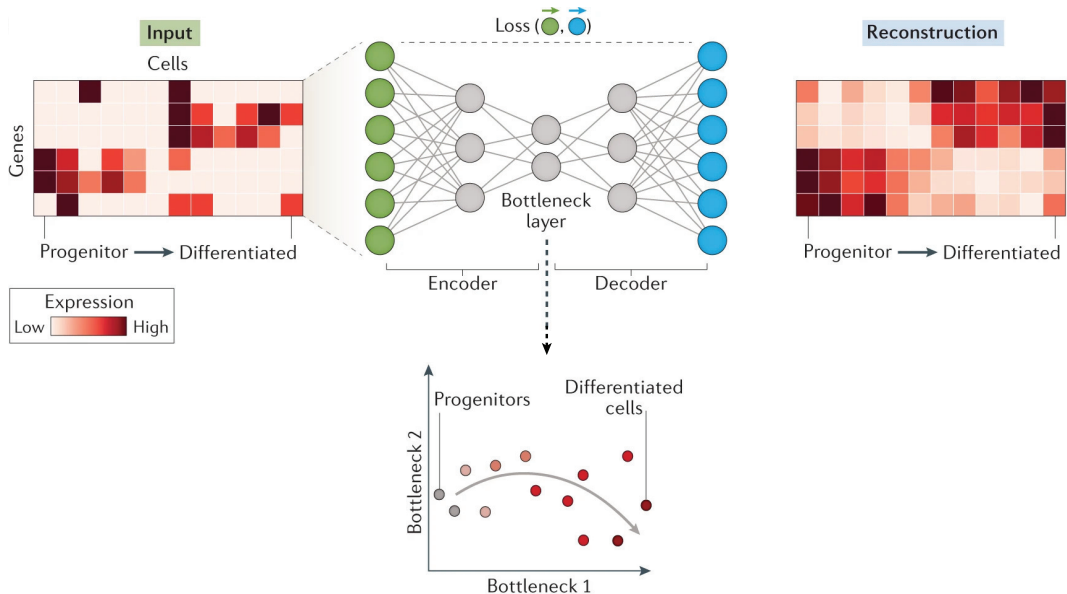


Discovering population structure from the genotype data

- It is often useful to reduce the high dimensional data by projecting it to a lower dimensional subspace to capture the “essence” of the data.
- Each observed high-dimensional data $\mathbf{x}_n \in \mathbb{R}^D$ was generated by a set of hidden or unobserved low-dimensional latent factors $\mathbf{z}_n \in \mathbb{R}^K$.
- Below illustrates applying **Principal Component Analysis (PCA)** to 1000 human genomes, each having 1 million SNPs



Non-linear dimensionality reduction by Autoencoder



Outline

Learning objectives

Unsupervised learning

K-means clustering

Principal component analysis

Autoencoders (optional)

- MLP autoencoder

- Denoising autoencoder

- CNN autoencoder

Evaluating unsupervised learning methods

Summary

Final exam (April 16 9-noon)

K-means clustering algorithm

Objective function:

$$J = \sum_{n=1}^N \left\| \mathbf{x}_n - \sum_{k=1}^K z_{n,k} \boldsymbol{\mu}_k \right\|^2 = \left\| \mathbf{X} - \mathbf{ZM} \right\|_F^2$$

where each row of $\mathbf{Z}$ is hot-encoding for the cluster assignment and $\mathbf{M} \in \mathbb{R}^{K \times D}$ are the K centroids for the D input features.

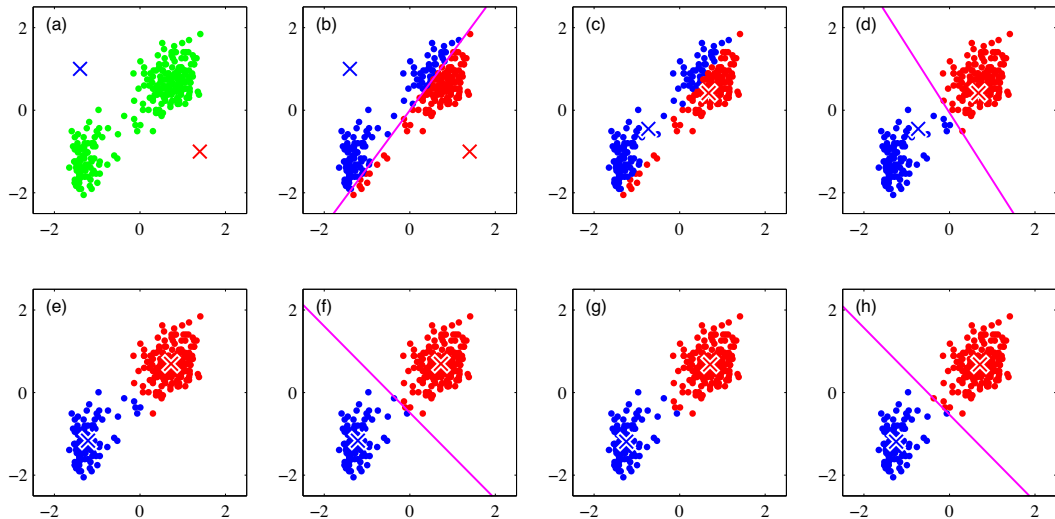
Algorithm 1 K-means clustering (ϵ)

- 1: Initialize K cluster centers $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$
- 2: **while** $J^{(t-1)} - J^{(t)} > \epsilon$ **do**
- 3: Assign each point $\mathbf{x}_n$ to the closest center k :

$$z_{n,k} = \begin{cases} 1 & \text{if } k = \arg \min_j \left\| \mathbf{x}_n - \boldsymbol{\mu}_j \right\|^2 \\ 0 & \text{if otherwise} \end{cases}$$

- 4: Update the cluster centroids: $\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_n z_{n,k} \mathbf{x}_n$, where $N_k = \sum_n z_{n,k}$.
- 5: **end while**

K-means clustering algorithm for $K=2$, $D=2$



Convergence on the cost function

- Cost function:

$$J = \sum_{n=1}^N \left\| \mathbf{x}_n - \sum_{k=1}^K z_{n,k} \boldsymbol{\mu}_k \right\|^2$$

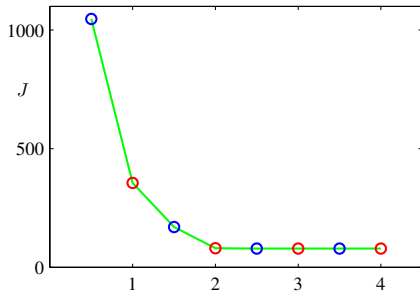
- K-means alternates between clustering and updating the K centers:

1. Clustering:

$$z_{n,k} = \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0 & \text{if otherwise} \end{cases}$$

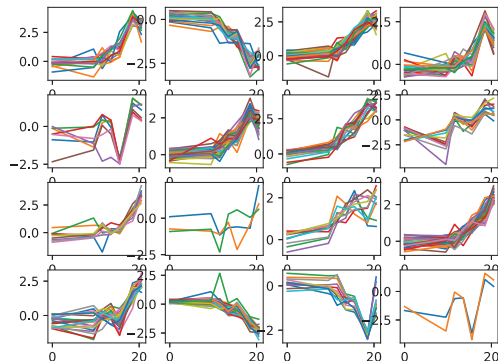
2. Updating:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N z_{n,k} \mathbf{x}_n, \quad \text{where} \quad N_k = \sum_{n=1}^N z_{n,k}$$

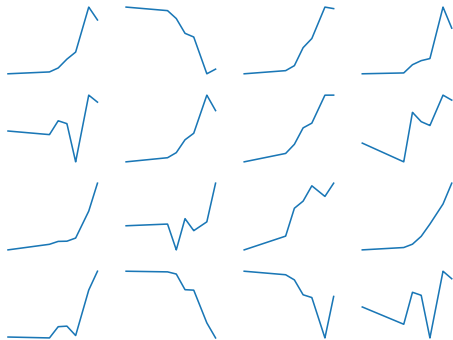


K-means clustering of the time-series yeast gene expression data

K-Means Clustering of Profiles



K-Means centroids



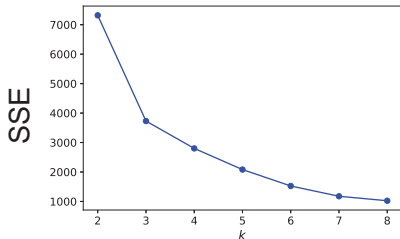
Murphy22 Figure 21.8

Evaluation metrics in choosing K

Reconstruction error on the validation set is not informative as it always decreases with increasing K :

$$J(K) = \sum_{n=1}^{N_{val}} \|\mathbf{x}_n^{(val)} - \sum_{k=1}^K z_{n,k}^{(val)} \hat{\boldsymbol{\mu}}_k\|^2$$

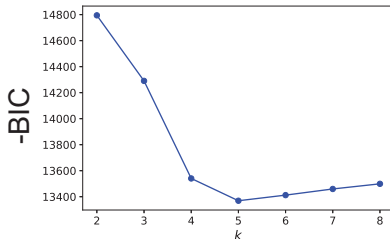
Validation error



Bayesian information criteria (BIC) score penalizes more complex model besides the reconstruction loss:

$$BIC(K) = -J(K) - \frac{DK}{2} \log(N)$$

Bayesian info criterion



Outline

Learning objectives

Unsupervised learning

K-means clustering

Principal component analysis

Autoencoders (optional)

- MLP autoencoder

- Denoising autoencoder

- CNN autoencoder

Evaluating unsupervised learning methods

Summary

Final exam (April 16 9-noon)

Principal Component Analysis

- Suppose we have an unlabelled dataset $\mathbf{X} \in \mathbb{R}^{D \times N}$ for D features and N examples.
- We would like to approximate each data point $\mathbf{x}_n \in \mathbb{R}^{D \times 1}$ by a low dimensional representation $\mathbf{z}_n \in \mathbb{R}^{K \times 1}$, where $K \leq D$.
- The variable $\mathbf{z}_n$ is known as the **latent factor**.
- The error produced by this approximation is called **reconstruction error**:

$$\begin{aligned}\mathcal{L}(\mathbf{W}) &= \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n; \mathbf{W}); \mathbf{W})\|_2^2 \\ &\triangleq \frac{1}{N} \|\mathbf{x}_n - \mathbf{W}\mathbf{z}_n\|_2^2\end{aligned}$$

where in the context of PCA:

- we assume a *linear* encoder and decoder
- $\mathbf{Z} \in \mathbb{R}^{K \times N}$ is also known as the **loading matrix**.
- $\mathbf{W} \in \mathbb{R}^{D \times K}$ is called the **basis matrix**. It is *orthogonal* matrix: $\mathbf{W}^\top \mathbf{W} = \mathbf{I}$, which means that $\mathbf{w}_k^\top \mathbf{w}_{k'} = 1$ for $k = k'$ or 0 for $k \neq k'$.

PCA derivation for the first PC

Let $\mathbf{z}_1 \in \mathbb{R}^{N \times 1}$ and $\mathbf{w}_1 \in \mathbb{R}^{D \times 1}$ be the loading and basis vector of the first PC. The reconstruction error is

$$\begin{aligned}\mathcal{L}(\mathbf{w}_1, \mathbf{z}_1) &= \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - z_{1,n} \mathbf{w}_1\|^2 = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - z_{1,n} \mathbf{w}_1)^\top (\mathbf{x}_n - z_{1,n} \mathbf{w}_1) \\ &= \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n^\top \mathbf{x}_n - 2z_{1,n} \mathbf{w}_1^\top \mathbf{x}_n + z_{1,n}^2 \underbrace{\mathbf{w}_1^\top \mathbf{w}_1}_1) = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n^\top \mathbf{x}_n - 2z_{1,n} \mathbf{w}_1^\top \mathbf{x}_n + z_{1,n}^2) \\ \frac{\partial \mathcal{L}(\mathbf{z}_1)}{\partial z_{1,n}} &= \frac{1}{N} (-2\mathbf{w}_1^\top \mathbf{x}_n + 2z_{1,n}) \stackrel{\Delta}{=} 0 \quad \Rightarrow \quad z_{1,n} = \mathbf{w}_1^\top \mathbf{x}_n \\ \mathcal{L}(\mathbf{w}_1) &= \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n^\top \mathbf{x}_n - 2z_{1,n} \mathbf{w}_1^\top \mathbf{x}_n + z_{1,n}^2) = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n^\top \mathbf{x}_n - 2z_{1,n}^2 + z_{1,n}^2) = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n^\top \mathbf{x}_n - z_{1,n}^2) \\ &\propto -\frac{1}{N} \sum_{n=1}^N z_{1,n}^2 = -\frac{1}{N} \sum_{n=1}^N \mathbf{w}_1^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_1 = -\mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 \quad \text{where} \quad \hat{\Sigma} = \frac{1}{N} \mathbf{X} \mathbf{X}^\top\end{aligned}$$

We see that minimizing the reconstruction error is equivalent to maximizing the variance of the latent representation w.r.t. $\mathbf{w}_1$ (since $\text{Var}[z] = \mathbb{E}[z^2] - \mathbb{E}^2[z] = \mathbb{E}[z^2]$, where $\mathbb{E}[z] = \mathbf{w}_1^\top \mathbb{E}[\mathbf{x}] = 0$).

The first PC is the eigenvector of $\hat{\Sigma}$ with the largest eigenvalue

Because we want the projection to be orthonormal such that $\mathbf{w}_1^\top \mathbf{w}_1 = 1$, we introduce the following constraint with the *Lagrange multiplier* λ_1 to the loss function:

$$\begin{aligned}\tilde{\mathcal{L}}(\mathbf{w}_1) &= -\mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 + \lambda_1 (\mathbf{w}_1^\top \mathbf{w}_1 - 1) \\ \frac{\partial \tilde{\mathcal{L}}(\mathbf{w}_1)}{\partial \mathbf{w}_1} &= -2\hat{\Sigma} \mathbf{w}_1 + 2\lambda_1 \mathbf{w}_1 \stackrel{\Delta}{=} 0 \quad \Rightarrow \quad \hat{\Sigma} \mathbf{w}_1 = \lambda_1 \mathbf{w}_1\end{aligned}\tag{1}$$

Therefore, the optimal solution for $\mathbf{w}_1$ is an *eigenvector* of $\hat{\Sigma}$ and λ_1 corresponds to the *eigenvalue*. Multiplying $\mathbf{w}_1^\top$ on both side, we have

$$\hat{\Sigma} \mathbf{w}_1 = \lambda_1 \mathbf{w}_1 \quad \Rightarrow \quad \mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 = \lambda_1 \mathbf{w}_1^\top \mathbf{w}_1 \quad \Rightarrow \quad \mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 = \lambda_1$$

Since we want to maximize $\mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1$ (i.e., minimizing the loss in Eq (1)), we pick the eigenvector that corresponds to the *largest* eigenvalue:

$$\mathbf{w}_1^* \leftarrow \arg \max_{\mathbf{w}_1} \mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 = \arg \max_{\mathbf{w}_1} \text{Var}[\mathbf{z}_1] = \arg \min_{\mathbf{w}_1} \mathcal{L}(\mathbf{w}_1)$$

Computing the second PC

We can find the second PC to further minimize the reconstruction error:

$$\begin{aligned}\mathcal{L}(\mathbf{w}_2) &= \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - z_{1,n}\hat{\mathbf{w}}_1 - z_{2,n}\mathbf{w}_2\|^2 = \frac{1}{N} \sum_{n=1}^N \|\tilde{\mathbf{x}}_n - z_{2,n}\mathbf{w}_2\|^2 \\ &\propto -\mathbf{w}_2^\top \hat{\Sigma} \mathbf{w}_2 \quad \text{where} \quad \hat{\Sigma} = \frac{1}{N} \tilde{\mathbf{X}} \tilde{\mathbf{X}}^\top\end{aligned}$$

Adding the orthogonal constraints $\mathbf{w}_1^\top \mathbf{w}_2 = 0$ and orthonormal constraint $\mathbf{w}_2^\top \mathbf{w}_2 = 1$:

$$\begin{aligned}\tilde{\mathcal{L}}(\mathbf{w}_2) &= -\mathbf{w}_2^\top \hat{\Sigma} \mathbf{w}_2 + \lambda_2 (\mathbf{w}_2^\top \mathbf{w}_2 - 1) + \lambda_{12} \mathbf{w}_2^\top \mathbf{w}_1 \\ \frac{\partial \tilde{\mathcal{L}}(\mathbf{w}_2)}{\partial \mathbf{w}_2} &= -2\hat{\Sigma} \mathbf{w}_2 + 2\lambda_2 \mathbf{w}_2 + \lambda_{12} \mathbf{w}_1 \stackrel{\Delta}{=} 0\end{aligned}$$

Solving for $\mathbf{w}_2$:

$$\begin{aligned}2\hat{\Sigma} \mathbf{w}_2 &= 2\lambda_2 \mathbf{w}_2 + \lambda_{12} \mathbf{w}_1 \quad \Rightarrow \quad 2\mathbf{w}_2^\top \hat{\Sigma} \mathbf{w}_2 = 2\lambda_2 \mathbf{w}_2^\top \mathbf{w}_2 + \lambda_{12} \mathbf{w}_2^\top \mathbf{w}_1 \\ \Rightarrow \mathbf{w}_2^\top \hat{\Sigma} \mathbf{w}_2 &= \lambda_2 \quad \Rightarrow \quad \hat{\Sigma} \mathbf{w}_2 = \lambda_2 \mathbf{w}_2\end{aligned}$$

Therefore, the solution for $\mathbf{w}_2$ for the second PC is the second largest eigenvector.

Generalizing to computing all K PCs

Find the k^{th} PC can be done in the same way

$$\mathcal{L}(\mathbf{w}_k) = \frac{1}{N} \sum_{n=1}^N \left\| \mathbf{x}_n - \sum_{s=1}^{k-1} z_{s,n} \mathbf{w}_s - z_{k,n} \mathbf{w}_k \right\|^2 = \frac{1}{N} \sum_{n=1}^N \left\| \tilde{\mathbf{x}}_n - z_{k,n} \mathbf{w}_k \right\|^2$$

Iteratively computing the k^{th} eigenvector is slow.

For $N \ll D$ (e.g., $N = 100$ samples versus $D = 20,000$ features), we can efficiently compute all eigenvectors by solving the *eigendecomposition* of the square and symmetric covariance matrix:

$$\underbrace{\mathbf{X}^\top \mathbf{X}}_{N \times N} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^{-1}$$

where $\mathbf{U}$ contains all of the N eigenvectors and $\mathbf{\Lambda}$ is the diagonal matrix with the diagonal elements being the eigenvalues. Because $\mathbf{U}$ is a orthogonal matrix, $\mathbf{U}^\top \mathbf{U} = \mathbf{I}$ and $\mathbf{U}^\top = \mathbf{U}^{-1}$.

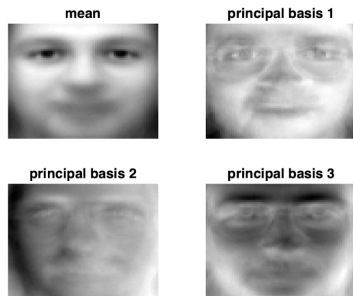
More efficiently, we can compute the truncated Singular Vector Decomposition (SVD) to get only the first $K < \min(N, D)$ PCs by $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^\top$ (details omitted).

Eigen faces

PCA were performed on 64×64 pixel images from the Olivetti face database (panel a). Mean and the first 3 PCA components $\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3$ are displayed in panel b.



(a)



(b)

Murphy22 Figure 20.3

Outline

Learning objectives

Unsupervised learning

K-means clustering

Principal component analysis

Autoencoders (optional)

- MLP autoencoder

- Denoising autoencoder

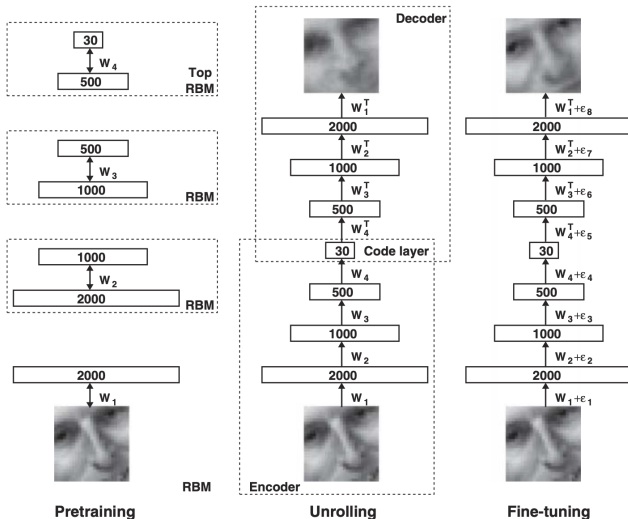
- CNN autoencoder

Evaluating unsupervised learning methods

Summary

Final exam (April 16 9-noon)

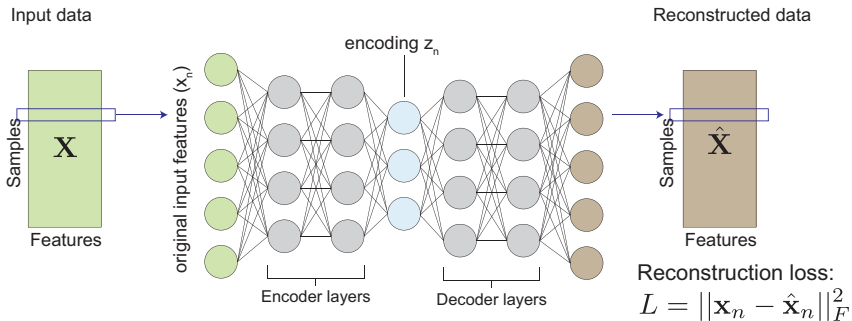
Autoencoder (AE) (Hinton & Salakhutdinov, Science 2006)



- Historically deep AE needs pre-training a stack of Restricted Boltzmann Machine (RBMs) with *contrastive divergence*:

$$\Delta w_{ij} = \epsilon (\langle v_i h_j \rangle_{data} - \langle v_i h_j \rangle_{recon})$$
- The stacks of RBMs are then unrolled to reconstruct the images and fine-tuned by backprop
- This type of architecture is popular in early 2000s as it helped mitigate vanishing gradients
- With advance in deep learning techniques such as ReLU, batch normalization, optimizer such as Adams, etc, RBM-based AE is less common these days.

Autoencoder end-to-end training



Encoding by a 3-layer feedforward network (i.e., encoder):

$$\mathbf{z}_n = f(f(\mathbf{x}_n \mathbf{W}_E^{(0)}) \mathbf{W}_E^{(1)}) \mathbf{W}_E^{(2)}$$

Decoding by another 3-layer feedforward network (i.e., decoder):

$$\hat{\mathbf{x}}_d = f(f(\mathbf{z}_n \mathbf{W}_D^{(0)}) \mathbf{W}_D^{(1)}) \mathbf{W}_D^{(2)}$$

$$\text{Loss: } L = \frac{1}{N} \sum_n ||\mathbf{x} - \hat{\mathbf{x}}_n||^2; \quad \text{Backpropagation: } \mathbf{W}_{\cdot}^{(\ell)} \leftarrow \mathbf{W}_{\cdot}^{(\ell)} - \epsilon \nabla L(\mathbf{W}_{\cdot}^{(\ell)})$$

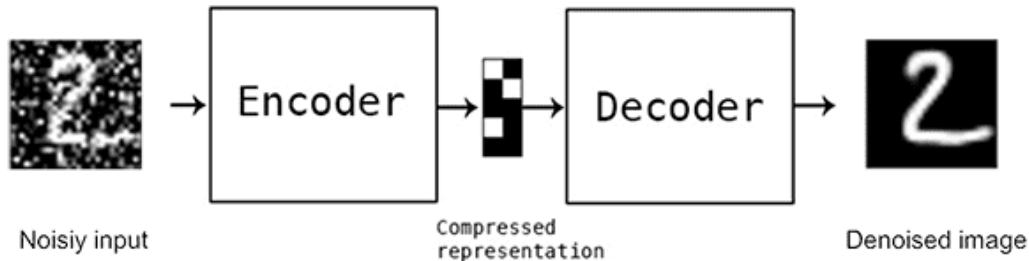
Denoising autoencoder (DAE)

DAE takes the original input with added Gaussian or Bernoulli noise (for binary image):

$$\tilde{\mathbf{x}} \sim \mathcal{N}(\mathbf{x}, \sigma^2 \mathbf{I}), \quad \text{or} \quad \tilde{\mathbf{x}} \sim \mathbf{x} \mathcal{B}(p, 1 - p)$$

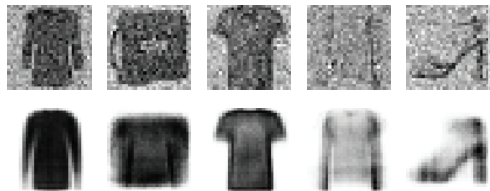
DAE are an extension of simple autoencoders to help:

- The hidden layers of the autoencoder learn more robust filters
- Reduce the risk of overfitting in the autoencoder
- Prevent the autoencoder from learning a simple identity function



Reconstructed Fashion MNIST images from validation set by DAE

with Gaussian noise

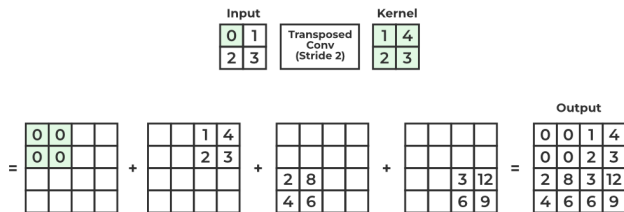
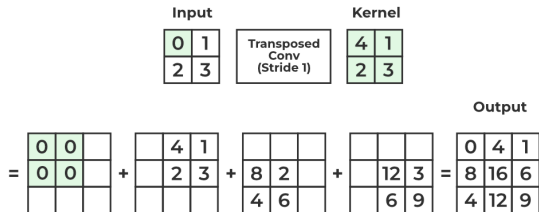


with Bernoulli dropout noise



(Murphy22 Chapter 20.3 Figure 20.19)

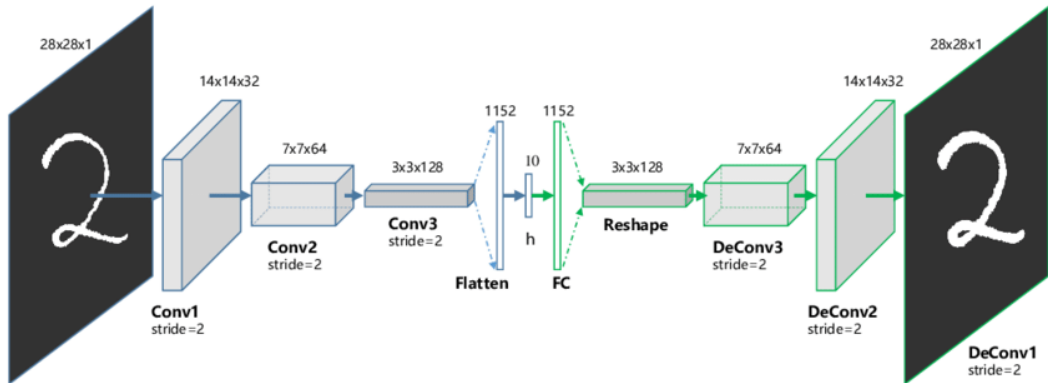
Transposed convolutional aka Deconvolution layer



- Instead of sliding the kernel over the input pixels and performing element-wise multiplication and summation, a transposed convolutional layer **slides the input pixel over the kernel** and performs element-wise multiplication and summation.
- This results in an output that is larger than the input, and the size of the output can be controlled by the stride and padding parameters of the layer.

([source](#))

CNN autoencoder



[source](#)

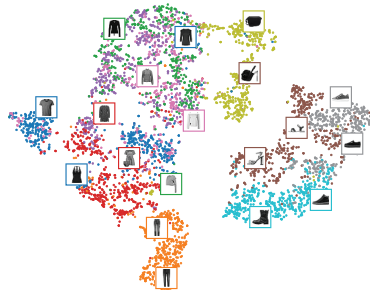
MLP AE vs CNN AE on Fashion MNIST (Murphy22 Fig 20.17 & 18)

MLP AE (784-100-30)



CNN AE

3x3 Conv (16), MaxPool (2x2), Conv (32, 3x3),
MaxPool (2x2), Conv (64, 3x3), MaxPool (2x2)



Outline

Learning objectives

Unsupervised learning

K-means clustering

Principal component analysis

Autoencoders (optional)

- MLP autoencoder

- Denoising autoencoder

- CNN autoencoder

Evaluating unsupervised learning methods

Summary

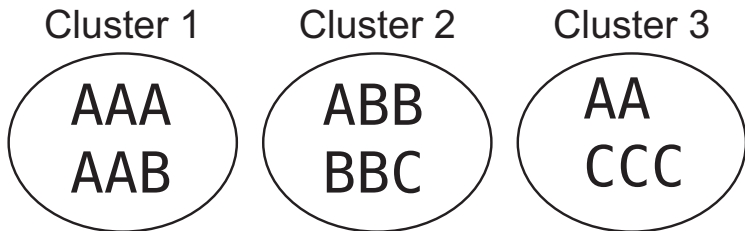
Final exam (April 16 9-noon)

Adjusted Rand Index (ARI) for evaluating clustering

We cluster N samples into R clusters or $\mathcal{U} = \{u_1, \dots, u_R\}$. Let set $\mathcal{V} = \{v_1, \dots, v_C\}$ be the ground-truth clusters. We can evaluate $\mathcal{U}$ based on $\mathcal{V}$ using ARI:

$$RI = \frac{TP + TN}{\binom{N}{2}}, \quad ARI = \frac{RI - \mathbb{E}[RI]}{1 - \mathbb{E}[RI]}$$

- TP : # pairs in *both* $\mathcal{U}$ and $\mathcal{V}$: $\binom{5}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2} = 40$
- TN : # pairs in *different* clusters in both $\mathcal{U}$ and $\mathcal{V}$
- $\mathbb{E}[RI]$ is the expected RI based on random clusterings that preserve the original number of classes and objects



Normalized mutual information (NMI)

- Proportion of common elements in cluster $u_i \subset U$ and cluster $v_j \subset V$:

$$p_{UV}(i, j) = \frac{|u_i \cap v_j|}{N}$$

where $|s|$ denotes the number of elements in the set.

- Proportion of elements in u_i and v_j :

$$p_U(i) = |u_i|/N, \quad p_V(j) = |v_j|/N$$

- Mutual information between U and V is defined as:

$$\mathbb{I}(U, V) = \sum_{i=1}^R \sum_{j=1}^C p_{UV}(i, j) \log \frac{p_{UV}(i, j)}{p_U(i)p_V(j)}$$

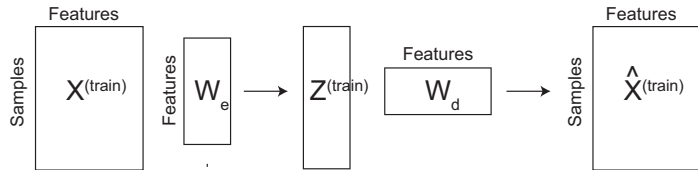
- Normalized mutual information:

$$NMI(U, V) = \frac{\mathbb{I}(U, V)}{(\mathbb{H}(U) + \mathbb{H}(V))/2}$$

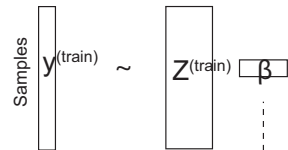
where entropy $\mathbb{H}(S) = -\sum_k p_S(k) \log p_S(k)$.

Unsupervised learning + Supervised learning evaluation

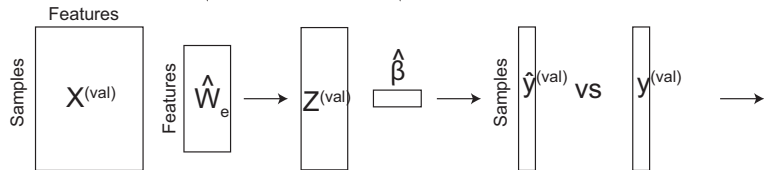
Training unsupervised learning model



Training supervised learning model



Validation



Outline

Learning objectives

Unsupervised learning

K-means clustering

Principal component analysis

Autoencoders (optional)

- MLP autoencoder

- Denoising autoencoder

- CNN autoencoder

Evaluating unsupervised learning methods

Summary

Final exam (April 16 9-noon)

Summary

- **Goal:** Discover latent structure in unlabeled data
- **Key Methods:**
 - **K-means Clustering** – Simple partitioning based on similarity
 - **Principal Component Analysis (PCA)** – Linear dimensionality reduction via variance maximization
 - **t-SNE** – Nonlinear embedding for high-dimensional visualization
 - **Autoencoders** – Neural networks that learn compact representations
 - MLP, Denoising, and CNN variants
- **Evaluation:**
 - **Clustering Metrics:** ARI and supervised learning with groundtruth labels
 - **Reconstruction Loss:** Assesses fidelity of representation
 - **Downstream Supervised Tasks:** Discover latent patterns from learned representations

Outline

Learning objectives

Unsupervised learning

K-means clustering

Principal component analysis

Autoencoders (optional)

- MLP autoencoder

- Denoising autoencoder

- CNN autoencoder

Evaluating unsupervised learning methods

Summary

Final exam (April 16 9-noon)

Final exam coverage and format

- Time: 9-noon on April 16
- Location: McGill Sports Complex Field House 35-44

Final exam covers *everything* in Modules 1-7 with more emphasis on the post-midterm materials (i.e., Module 4.5 - Module 7).

1. You have 180 minutes to write the exam.
2. Hard-copy notes and books are allowed but electronics are not allowed.
3. This exam contains 25 questions on 18 pages for total of 100 points (with the last two pages as scribble sheet).
4. 18 multiple-choice questions (54 pts, 3 pts each): answer them on a scantron
5. 7 short answer questions (46 pts, 6+5+5+10+5+10+5): write your answer directly below each question.

Thank you all! Good luck with the final! 😊